

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Web Programlama

React I: Bileşenler

Hafta 4

React I: Bileşenler

Doç. Dr. Tolga Berber

KTÜ Fen Fakültesi, Bilgisayar Bilimleri Bölümü · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Hafta 3'te bir koltuğa tıklandığında hem koltuğun kendi rengini hem de sayfanın başka bir yerindeki "seçili koltuk" metnini ayrı ayrı, elle güncellemeniz gerekiyordu.

- Bu iki güncellemeden birini unutsaydınız ekran, gerçek durumdan farklı bir şey gösterirdi — kullanıcı "seçili" sandığı bir koltuğun aslında seçili görünmediğini fark ederdi
- Bu hafta "durumu tanımlayın, görünümü kendisi güncellesin" fikrini (React) tanıtarak bu senkronizasyon yükünü ortadan kaldırıyorsunuz
- Etkinlik/Bilet Sisteminin ön yüzü bugünden itibaren TSX bileşenlerinden kurulacak — veri hâlâ Hf.3'ün sabit `veri.ts` dizileri

YG I/II'de bir fonksiyona `const T&` geçirme alışkanlığını ve C++'ın `const` disiplinini biliyorsunuz — React'i bu derste *ilk kez* göreceksiniz.

Bu Hafta

◎ Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

1. React'in bildirimsel modelini Hf.3'ün elle DOM kodundan farkıyla açıklamak, Vite `react-ts` ile proje kurup TSX yazmak
2. `props`/`interface` ile tiplenmiş, `children` kabul eden fonksiyon bileşenleri oluşturmak
3. `useState` ile bileşenin kendi durumunu tutmasını, değişmezlik ilkesiyle güncellemeyi ve olaylara tepki vermeyi uygulamak
4. `map`+`key` ile bileşen listesi üretmek, koşullu render kalıplarını kullanmak
5. Hf.3'ün elle DOM kodunu React bileşenlerine dönüştürerek Etkinlik/Bilet Sistemine ilk React katmanını eklemek

BÖLÜM 1

Ünite 1 — Neden Bileşen? React'e Giriş ve TSX

React Giriş props/Bileşen state/Olaylar Liste/Render Özet

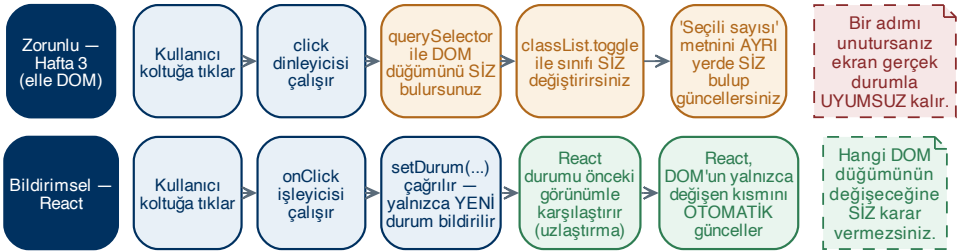
2 • İçerik Zorunlu Koddan Bildirimsel Modele

- Hf.3'te `querySelector` ile öğeyi bulup `classList.toggle` ile elle güncellediniz — bu zorunlu programlamadır: DOM'u nasıl güncelleyeceğinizi siz yazarsınız
- React'te yalnızca "durum şu anda böyleyse görünüm böyle olsun" tanımlarsınız — nasıl yapılacağına React karar verir: bildirimsel programlama
- Sanal DOM: React her güncellemede önce hafif bir taslak ağaç hesaplar; uzlaştırma yalnızca farkı gerçek DOM'a uygular (fikir düzeyinde — Ünite 3-4'te somutlaşır)

Tanım: Bildirimsel programlama / Zorunlu programlama

Zorunlu programlamada "nasıl yapılacağı" adım adım kodla belirtilir; bildirimsel programlamada yalnızca "ne" istendiği tanımlanır, güncellemenin nasıl yapılacağını kütüphane (React) belirler.

Zorunlu vs Bildirimsel: Yan Yana



Her iki akış da AYNI kullanıcı tıklamasından başlar;
fark, görünümü kimin güncellediğidir.

Analoji: Restoran Siparişi ve Taslak Düzeltme

Analoji

Bu ayrımı bir restoranda sipariş vermekle mutfağa girip yemeği kendiniz pişirmeye benzetin: zorunlu yaklaşımda mutfağa girip her adımı siz yaparsınız; bildirimsel yaklaşımda garsona yalnızca ne istediğinizi söylersiniz — nasıl pişirileceğine mutfak (React) karar verir.

Analoji

Sanal DOM ve uzlaştırmayı bir taslak üzerinde düzeltme yapıp yalnızca değişen satırları temize çekmeye benzetin: tüm belgeyi baştan yazmak yerine, önce bir taslak hazırlanır, eskisiyle karşılaştırılır, yalnızca farklı kısımlar temiz kâğıda (gerçek DOM'a) geçirilir.

Vite `react-ts` ile Proje Kurulumu

```
>_ Terminal

$ npm create vite@latest istemci -- --template react-ts
$ cd istemci && npm install
$ npm run dev
```

- Hf.3'ün `vanilla-ts` şablonuna şaşırtıcı derecede benzer: yine `index.html`, `src/main.*` (artık `.tsx`), `tsconfig.json`, `package.json`
- Fark: `package.json`'a `react`/`react-dom` bağımlılıkları, `vite.config.ts`'e `@vitejs/plugin-react` eklenir
- `npm run dev` / `npm run build` ayrımı Hf.3'teki gibi aynı kalır — geliştirme sunucusu üretimde asla kullanılmaz

Kod: TSX'i Derleyen Eklenti — `vite.config.ts`

```ts
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vite';

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
});
```

Bu eklenti Vite'a TSX/JSX'i derlerken React'in kendine özgü dönüşümünü (JSX'i fonksiyon çağrılarına çevirme) uygulamasını söyler.

TSX Sözdizimi: İfade Gömme, `className`, Fragment

- İfade gömme: `{degisken}`, `{1 + 1}` — yalnızca bir değer üreten ifadeler; `if`/`for` gibi bir deyim süslü parantez içine yazılamaz
- `class` yerine `className` — `class` JavaScript'te zaten ayrılmış bir sözcüktür
- Bir bileşen tek bir kök öğe döndürür; birden fazla kardeş öğe için ekstra bir `<div>` istemiyorsanız görünmez bir Fragment (`<>...</>`) kullanılır

```
// YANLIŞ: iki kök öğe doğrudan döndürülemez
return (
  <h1>Hoşgeldiniz</h1>
  <p>Etkinlik sayısı: 3</p>
);
```

BÖLÜM 2

Ünite 2 — props ve Tiplenmiş Bileşen

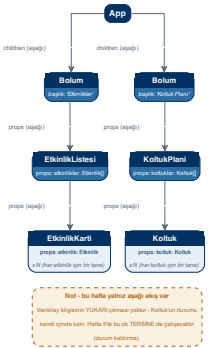
2 • İçerik Fonksiyon Bileşeni, `props`, `interface`, `children`

- Fonksiyon bileşeni: JSX döndüren sıradan bir TypeScript fonksiyonu — büyük harfle başlamalı (küçük harfli ad HTML etiketi sanılır)
- `props`: bir bileşene dışarıdan veri geçirmenin tek yolu — parametre geçirmeye benzer ama salt okunurdur
- `interface XProps`: bileşenin aldığı `props`'un "şeklini" tanımlar — yanlış/eksik prop `tsc` hatası verir
- `children`: açılış/kapanış etiketleri arasına yazılan JSX otomatik olarak `props.children` olarak gelir

Tanım: props

Bir bileşene dışarıdan geçirilen, bileşenin kendisi tarafından değiştirilmeyen salt okunur veridir; yalnızca üstten alta doğru akar.

Bileşen Ağacı ve Tek Yönlü Veri Akışı



Analoji: Form Dilekçesi ve Hediye Kutusu

Analoji

`props`'u bir form dilekçesine önceden doldurulmuş bilgiler vermeye benzetin: dilekçeyi (bileşeni) dolduran kişi (üst bileşen) hangi bilgilerin gireceğine karar verir; dilekçenin kendisi bu bilgileri değiştiremez, yalnızca gösterir.

Analoji

`children`'ı bir hediye kutusunun içine istediğiniz hediyeyi koymaya benzetin: kutunun (sarmalayıcı bileşenin) tasarımı sabittir; içine hangi hediyenin (içeriğin) konacağı her kullanımda değişir — tıpkı bir yapı setinin (LEGO) aynı parçasının farklı renklerle tekrar kullanılması gibi.

Kod: `children` ile Sarmalayıcı — `Bolum.tsx`

```tsx
// children örneği: Bolum, açılış/kapanış etiketleri arasına konan
// HERHANGİ bir JSX'i props.children olarak alıp bir başlığın altında
// gösteren küçük bir sarmalayıcı bileşendir.
interface BolumProps {
  baslik: string;
  children: ReactNode;
}

export function Bolum({ baslik, children }: BolumProps) {
  return (
    <section className="bolum">
      <h2>{baslik}</h2>
      {children}
    </section>
  );
}
```

`children: ReactNode` tipi, açılış/kapanış etiketleri arasına yazılan herhangi bir JSX'i kabul eder.

Kod: `props` ile Tiplenmiş Bileşen — `EtkinlikKarti.tsx`

```tsx
// Hf.2'nin article.etkinlik-karti HTML yapısının React karşılığı;
// biletFiyati 0 ise üçlü operatörle "Ücretsiz" rozeti gösterilir.
export function EtkinlikKarti({ etkinlik }: EtkinlikKartiProps) {
  return (
    <article className="etkinlik-karti">
      <h3>{etkinlik.baslik}</h3>
      <p>
        {etkinlik.tarih} · {etkinlik.mekan}
      </p>
      <p>
        Kapasite: {etkinlik.kapasite} ·{' '}
        {etkinlik.biletFiyati === 0 ? (
          <span className="rozet-ucretsiz">Ücretsiz</span>
        ) : (
          `Bilet: ${etkinlik.biletFiyati} TL`
        )}
      </p>
    </article>
  );
}
```

`interface EtkinlikKartiProps` yalnızca `etkinlik: Etkinlik` alanı bekler; fiyat 0 ise “Ücretsiz” rozeti gösterilir.

BÖLÜM 3

Ünite 3 — state ve Olaylar

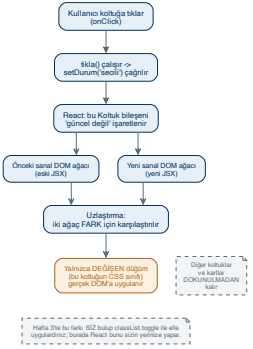
2 • İçerik useState, Değişmezlik İlkesi, Olay İşleyicileri

- `useState`: bir bileşenin render'lar arasında hatırlanan kendi verisini (durumunu) tutmasını sağlayan React "hook"u
- `const [durum, setDurum] = useState<KoltukDurumu>(deger)`: mevcut değer + onu değiştirmenin tek yolu; `onClick` gibi olay işleyicileri genellikle `setDurum(...)` çağırır
- Değişmezlik ilkesi: eski değeri doğrudan değiştirmek yerine yeni bir değer üretip `setX(...)`'e verirsiniz — aksi halde React değişikliği fark etmez

Tanım: state (durum)

Bir bileşenin render'lar arasında hatırladığı, yalnızca kendi kontrolündeki veridir; başka hiçbir bileşen bu değeri doğrudan göremez/değiştiremez.

`useState` → Yeniden Render → Uzlaştırma



Analoji: Işık Düğmesi ve Sözleşme Nüshası

Analoji

`state`'i bir ışık düğmesinin kendi konumunu (açık/kapalı) hatırlamasına benzetin: düğmeye her bastığınızda önceki konumunu unutmaz, yeni konumunu tutar — bir bileşenin `useState`'i de aynı şekilde en son değerini hatırlar.

Analoji

Değişmezlik ilkesini bir sözleşmenin eski nüshasını çizip üstüne yazmak yerine yeni, temiz bir nüsha hazırlamaya benzetin: eski nüsha bozulmadan bir kenarda kalır, imzalanan (React'e bildirilen) her zaman yeni nüshadır — bu sayede "hangi nüsha değişti" karşılaştırması net yapılabilir.

Kod: `Koltuk` Bileşeni — `state` ve Tıklama

```tsx
// Koltuğun kendi durumu, koltuk.durum başlangıç değeriyle yerel bir
// useState'te tutulur; App ya da KoltukPlani bu değeri GÖRMEZ.
const [durum, setDurum] = useState<KoltukDurumu>(koltuk.durum);

function tikla() {
  if (durum === 'dolu') return;
  setDurum((onceki) => (onceki === 'bos' ? 'secili' : 'bos'));
}
```

`dolu` durumunda `tikla` erken çıkış yapar; aksi hâlde `setDurum` eski değeri okuyup yeni bir değer döndürür — mutasyon yok.

Kod: `Koltuk` Bileşeni — Render ve Koşullu Rozet

```jsx
return (
  <button
    type="button"
    className={`koltuk ${durum}`}
    onClick={tikla}
    disabled={durum === 'dolu'}
    aria-disabled={durum === 'dolu'}
  >
    {koltukEtiketi(koltuk)}
    {durum === 'dolu' && <span className="koltuk-rozet">Dolu</span>}
  </button>
);
```

`disabled`/`aria-disabled` `dolu` koltuğu tıklanamaz yapar; `&&` ile “Dolu” rozeti — Ünite 4’ün koşullu render kalıbının ilk örneği.

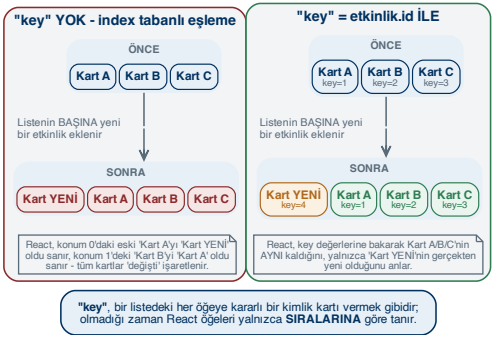
BÖLÜM 4

Ünite 4 — Liste ve Koşullu Render

2 • İçerik map, key, Koşullu Render Kalıpları

- `map`: bir diziden JSX öğe dizisi üretir — Hf.3'ün `forEach`+`createElement` döngüsünün bildirimsel karşılığı
- `key`: her öğeye veriden gelen kararlı bir kimlik (`etkinlik.id`) — React'in uzlaştırmada "hangisi değişti" sorusunu doğru cevaplaması için zorunlu; `index` kullanılmaz
- Koşullu render: `&&` (küçük bir öğe eklemek), üçlü operatör (iki alternatiften biri), erken dönüş (`if (...) return ...;`)
- Boş liste: kullanıcıya boş bir sayfa yerine anlamlı bir mesaj gösterme alışkanlığı

key Olmadan / key ile Uzlaştırma



Analoji: Vestiyer Fişi ve Duruma Göre Değişen Tabela

Analoji

`key`'i bir vestiyerdeki numaralı fişlere benzetin: her paltoya (liste öğesine) sabit bir numara (id) verilirse, palto sırası karışsa bile doğru palto doğru numarayla eşleşir; numara yerine yalnızca "sıradaki kanca" (index) kullanılırsa, araya yeni bir palto girdiğinde tüm paltolar yanlış kancalarla eşleşmiş gibi görünebilir.

Analoji

Koşullu render'ı bir tabelanın duruma göre farklı yazı göstermesine benzetin: "stokta var" / "tükendi" tabelası mağazanın durumuna göre otomatik değişir; raf tamamen boşsa tabela ayrı bir mesaja ("Bu bölüm şu an boş") döner.

Kod: Etkinlik Listesi — `map` + `key`

```jsx
// map ile diziden bileşen listesi üretme; her karta veriden gelen
// kararlı bir key (etkinlik.id) verilir.
return (
  <div className="etkinlik-listesi">
    {etkinlikler.map((etkinlik) => (
      <EtkinlikKarti key={etkinlik.id} etkinlik={etkinlik} />
    ))}
  </div>
);
```

- Her karta veriden gelen kararlı bir `key` (`etkinlik.id`) verilir — `index` kullanılmaz
- Boş dizi durumu (`etkinlikler.length === 0`) bu koddan önce, erken dönüşle ayrı ele alınır

Kod: App Bileşeninde Birleştirme

```tsx
// veri.ts'ten gelen sabit dizileri, Bolum (children) ile sarmalanmış iki
// bölümde EtkinlikListesi/KoltukPlani'ya prop olarak geçirir.
export function App() {
  return (
    <>
      <Bolum baslik="Etkinlikler">
        <EtkinlikListesi etkinlikler={etkinlikler} />
      </Bolum>
      <Bolum baslik="Koltuk Planı">
        <KoltukPlani koltuklar={koltuklar} />
      </Bolum>
    </>
  );
}
```

`veri.ts`'ten gelen sabit diziler, `Bolum` (children) ile sarmalanmış iki bölümde `EtkinlikListesi`/`KoltukPlani`'ya prop geçirilir — Etkinlik/Bilet Sisteminin ilk React katmanı.

BÖLÜM 5

Özet

Özet

Özet

- Zorunlu/bildirimsel ayrımı + Sanal DOM/uzlaştırma + Vite `react-ts` + TSX: React'in temel modelini ve söz dizimini kavradınız
- `props` + `interface` + `children`: tiplenmiş, yeniden kullanılabilir fonksiyon bileşenleri oluşturdunuz
- `useState` + değişmezlik ilkesi + olay işleyicileri: bir bileşene kendi hafızasını kazandırıp kullanıcı etkileşimine tepki verdirdiniz
- `map` + `key` + koşullu render: dizilerden bileşen listesi üretip `App.tsx`'te Etkinlik/Bilet Sisteminin ilk React katmanını birleştirdiniz

5 • İzleme Aktarım — Başka Nerede?

- Bugün kurduğunuz `react-ts` iskeleti Hf.5, Hf.7 ve sonrasında aynı `istemci/` klasöründe büyümeye devam edecek — proje kurulumunu her hafta yeniden yapmayacaksınız
- `EtkinlikKarti`'nin aldığı `etkinlik: Etkinlik` prop'u, Hf.6'da FastAPI'den gelecek gerçek API verisiyle değişmeden kullanılmaya devam edecek
- Bu haftaki koltuk seçimi yalnızca `Koltuk` bileşeninin kendi içinde yaşıyor — Hf.5'te bu durumu bir üst bileşene taşıyarak (durum kaldırma) bir sepet oluşturacaksınız
- `EtkinlikListesi`/`KoltukPlani`'yı birleştirdiğiniz `App.tsx` kalıbı, Etkinlik/Bilet Sisteminin geri kalan tüm haftalarda büyüyeceği omurgadır

Sırada Ne Var?

Bu hafta lab'da

Hafta 3'ün elle DOM güncelleyen `istemci/` projesini React `react-ts` şablonuyla baştan kurup `EtkinlikKarti`, `EtkinlikListesi`, `Koltuk` ve `KoltukPlani` bileşenlerini kendi elinizle yazıp `npm run dev` ile tarayıcıda canlı test edeceksiniz.

- Kendinizi Sınayın her üniteyi kapsıyor: tek kök öğe kuralı, `props`'u doğrudan değiştirmenin neden kötü bir pratik olduğu, `durum = 'secili'` doğrudan ataması neden hata verir, `key` eklemeyi unutursanız ne olur
- Gelecek hafta: React II — Hooks, Form, Durum Paylaşımı — bu haftanın yerel `Koltuk` durumu bir üst bileşene taşınacak, `useEffect` ve formlarla gerçek bir sepet oluşturacaksınız